Module 16

Using GitHub Copilot with Python

Agenda

GitHub Foundations Part 1

1 Introduction to Git

2 Introduction to GitHub

3 Introduction to GitHub's products

4 Configure code scanning on GitHub

5 Introduction to GitHub Copilot

6 Code with GitHub Codespaces

7 Manage your work with GitHub Projects

8 Communicate effectively on GitHub using Markdown

GitHub Foundations Part 2

9 Contribute to an open-source project on GitHub

10 Manage an InnerSource program by using GitHub

11 Maintain a secure repository by using GitHub best practices

12 Introduction to GitHub administration

13 Authenticate and authorize user identities on GitHub

14 Manage repository changes by using pull requests on GitHub

15 Search and organize repository history by using GitHub

16 Using GitHub Copilot with Python

Introduction

In this module, you learn how you can use GitHub Copilot to modify a project by using a prompt to customize a Python API. You also learn how to use live suggestions after typing initial code.

By the end of this module, you'll be able to:

- Configure a GitHub repository in Codespaces and installed the GitHub Copilot extension.
- Craft prompts to generate suggestions from GitHub Copilot.
- Apply GitHub Copilot to improve your Python projects.

After successfully finishing this module, you're capable of using a prompt to customize a Python project with GitHub Copilot in GitHub Codespaces.

MCP

What is GitHub Copilot?

How GitHub Copilot Works

- **AI Assistant:** GitHub Copilot generates code and more based on natural language prompts typed within your IDE.
- **Prompts:** A natural language prompt that can generate code for you to use looks like:

```
# Create a web API using FastAPI
with a route to products
```

Recognizing Prompts

- **Comment in Code File:** Copilot recognizes prompts typed as comments in code files (e.g., .py, .js).
- **Markdown File:** Typing text in a markdown file and waiting a few seconds will also prompt Copilot to respond.

Accepting Suggestions

- **Gray Code:** *Suggestions* appear as gray code; press the **Tab** key to accept.
- **Multiple Suggestions:** Use **Ctrl + Enter** to cycle through and select the most appropriate suggestion.

Exercise - Set up GitHub Copilot to work with Visual Studio Code

In this exercise, we create a new repository using the GitHub template for the Python personal portfolio frontend web application.

To use GitHub Copilot, you need the following:

- **GitHub Account**
  - Copilot is a GitHub service, so you need a GitHub account to use it.
- **Check Copilot Access**
  - Check the Copilot pricing plans for your situation.
  - For learning, the **Copilot Free** option with usage limits should be sufficient.
  - It includes:
    - Access to inline code completions
    - Multi-file editing
    - Copilot Chat
    - Selecting multiple models
    - Support across all editors and in github.com

Exercise - Set up GitHub Copilot to work with Visual Studio Code

In this exercise, we create a new repository using the GitHub template for the Python personal portfolio frontend web application.

To use GitHub Copilot, you need the following:

GitHub Account

- Copilot is a GitHub service, so you need a GitHub account to use it.

Check Copilot Access

- Check the Copilot pricing plans for your situation.
- For learning, the **Copilot Free** option with usage limits should be sufficient.
- It includes:
  - Access to inline code completions
  - Multi-file editing
  - Copilot Chat
  - Selecting multiple models
  - Support across all editors and in github.com

Verify Copilot Access is Enabled:

- In GitHub, go to your profile, then head over to **Settings**.
- In the left navigation, select **Copilot**.
- Verify Copilot is active and **Copilot Chat in the IDE** is enabled.

Install the Extension:

- Extensions are available for major IDEs like Visual Studio, Visual Studio Code, JetBrains IDEs, VIM, and XCode.
- Simply search for `GitHub Copilot` in your IDE's extension marketplace and install it.

Exercise - Set up GitHub Copilot to work with Visual Studio Code (cont.)

In this exercise, we create a new repository using the GitHub template for the Python personal portfolio frontend web application.

Launch the Codespaces environment, which comes preconfigured with the GitHub Copilot extension.

1. Open the Codespace with the preconfigured environment in your browser.
2. On the **Create codespace** page, review the codespace configuration settings, then select **Create new codespace**.
3. Wait for the codespace to start. This startup process can take a few minutes.
4. The remaining exercises in this project take place in the context of this development container.

Python Web API
When complete, Codespaces loads with a terminal section at the bottom.

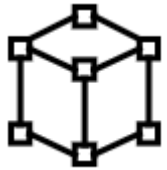
Codespaces installs all the required extensions in your container. Once the package installs are completed, Codespaces executes the `uvicorn` command to start your web application running within your Codespace.

When the web application successfully starts, a message in the terminal shows that the server is running on port 8000 within your Codespace.

Use GitHub Copilot with Python

Developing with GitHub Copilot

- **Continuous Updates:** Ensure code is fresh, fix bugs, and add new features using GitHub Copilot and Copilot Chat.
- **Interactive Chat:** Use Copilot Chat to ask and receive answers to code-related questions.



Prompt Engineering

- **Creating Prompts:** Build prompts with clear instructions to generate specific responses from Copilot.
- **Effective Prompts:** Crafting specific and clear prompts ensures high-quality output from Copilot.

Best Practices for Using Copilot

- **Simple to Elaborate:** Start with simple prompts and gradually add more details for specific suggestions.
- **Cycle Suggestions:** Use **Ctrl+Enter** to cycle through suggestions and select the best output.

Exercise - Update a Python web API with GitHub Copilot

In this exercise, you modify a Python repository using code suggestions from GitHub Copilot to create an interactive HTML form and an Application Programming Interface (API) endpoint.

Step 1: Add a Pydantic Model

- **Add Comment:** In the `main.py` file, add a comment to generate a `Pydantic` model with GitHub Copilot.
- **Generated Model:** The model should look like:

```
class Text(BaseModel):
text: str
```

Step 2: Generate a New Endpoint

- **Add Comment:**

```
# Create a FastAPI endpoint that accepts a POST request
with a JSON body containing a single field called
"text" and returns a checksum of the text
```

- **Generated Endpoint:** GitHub Copilot will generate the necessary endpoint code based on the comment.

Step 3: Add Necessary Imports

- **Missing Imports:** Ensure the base64 and os modules are imported to prevent application crashes.
- **Verify Endpoint:** Check the new endpoint by going to the **/docs** endpoint and confirming its presence.

Summary

You learned how to:

- Configure a GitHub repository in Codespaces and installed the GitHub Copilot extension.
- Craft prompts to generate suggestions from GitHub Copilot.
- Apply GitHub Copilot to improve your Python projects.

End of presentation